HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

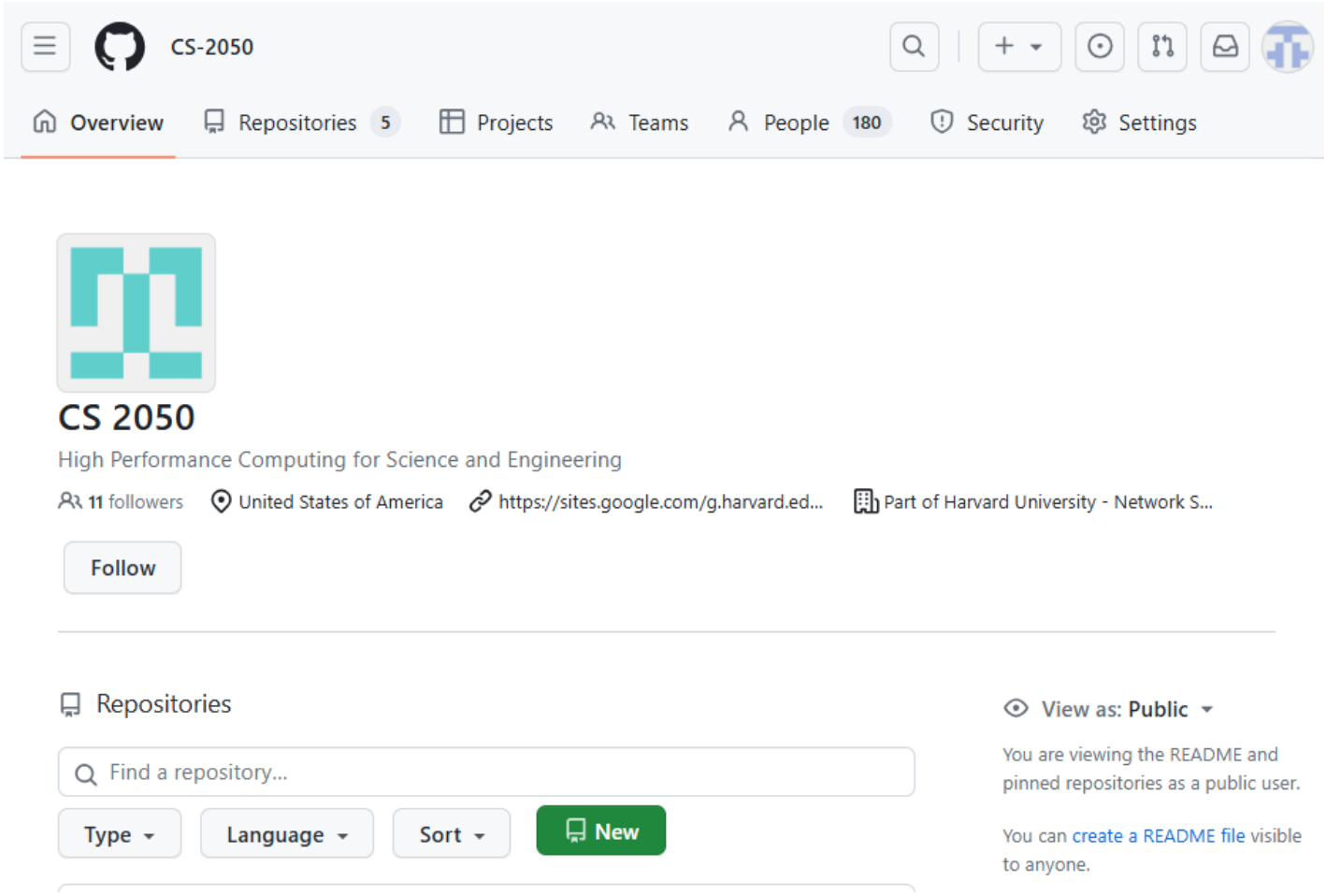
Harvard University
CS 2050

Lecture 4

February 5, 2026

CS-2050 GITHUB ORGANIZATION

- If you haven't used `code.harvard.edu` previously, please **log on now**
- Your account won't exist until you log on for the first time
- We can't add you to CS-2050 until your account exists



CS-2050 / homework-1

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights

homework-1

Internal template

Edit Pins | Watch 0 | Fork 0 | Star 0 | Use this template

main | Go to file | Code

laz380 and Chuck Witt public release. 0aab4ce · yesterday 1 Commits

question-1	public release.	yesterday
question-2	public release.	yesterday
question-3	public release.	yesterday
question-4	public release.	yesterday
question-5	public release.	yesterday
question-6	public release.	yesterday
question-7	public release.	yesterday
README.md	public release.	yesterday

About

No description, website, or topics provided.

Readme
Activity
Custom properties
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

SPACK

- Load the CS-2050 environment

```
[$]   . ~/161588/spack/share/spack/setup-env.sh
[$]   spack env activate -p CS-2050
```

CS-2050 / questions-and-answers

Type / to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

questions-and-answers

Public

Edit Pins | Watch 0 | Fork 0 | Star 0

main | Go to file t | Code

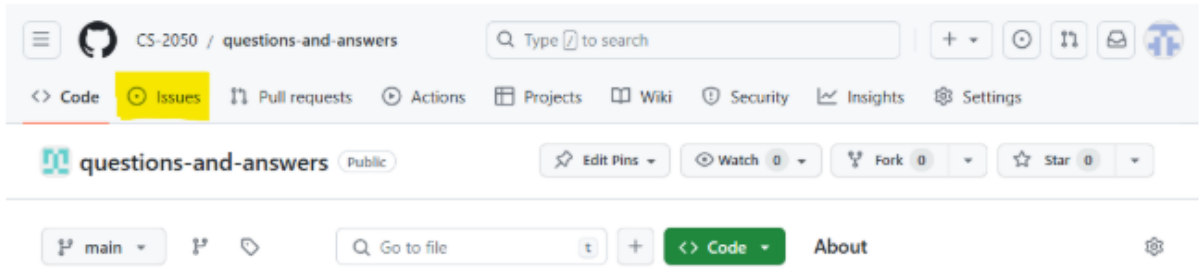
wcw398 update readme 4be70a2 · 17 hours ago 4 Commits

README.md update readme 17 hours ago

README

Questions and Answers

Please create issues in this repository to ask questions about the course.



About

Ask your CS 2050 questions here.

sites.google.com/g.harvard.edu/cs-2050

Readme

Activity

Custom properties

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

CMAKE: TWO STEPS

QUIZ 1 COMPLETE

- Configure

What does this dot mean?

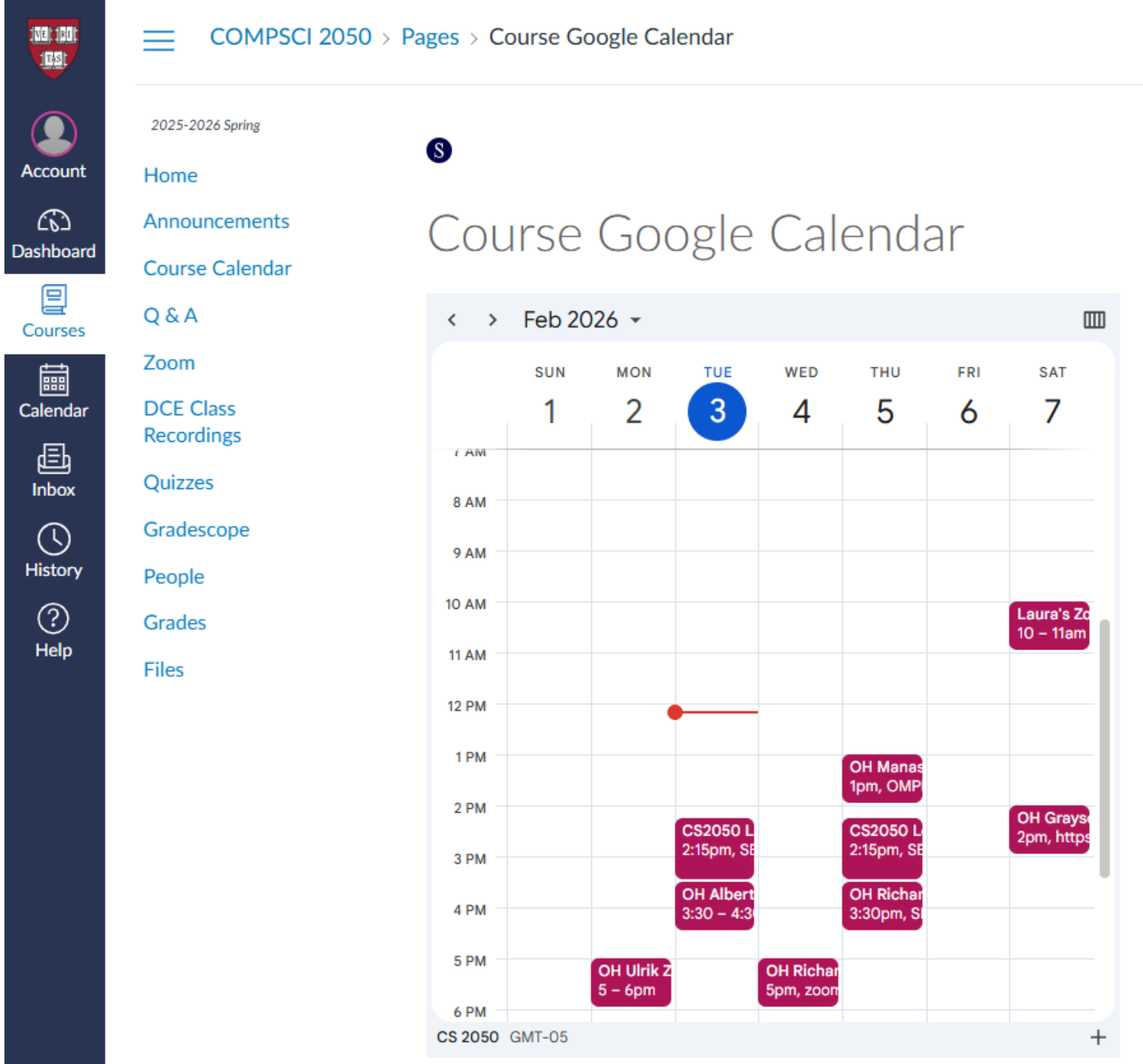
```
[$]  cmake -B build .
```

- Build

```
[$]  cmake –build build
```

OFFICE HOURS

- Temporary location: SEC, Room 1.312
- Or Zoom where indicated



DOWNLOAD EXAMPLES

```
[$] cd ~
[$] git clone https://code.harvard.edu/CS-2050/lecture-examples
[$] cd lecture-examples/lecture-04
```

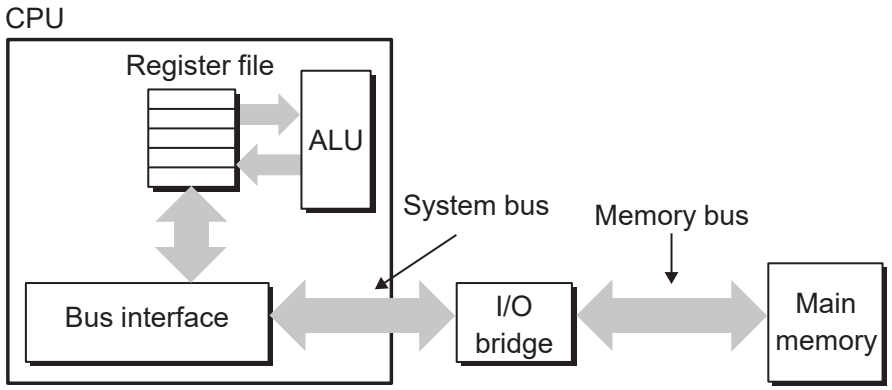
EXAMPLE 1

BITS AND BYTES

How many bits (and bytes) are used to store a double precision floating point number?

- 16 bits (2 bytes)
- 32 bits (4 bytes)
- 64 bits (8 bytes)
- 128 bits (16 bytes)

VON NEUMANN ARCHITECTURE



von Neumann architecture

John von Neumann

- Classic von Neumann architecture consists of:
  (1) main memory, (2) processor, and (3) interconnect
- Main memory stores both instructions and data, which are transferred through the interconnect (bus)

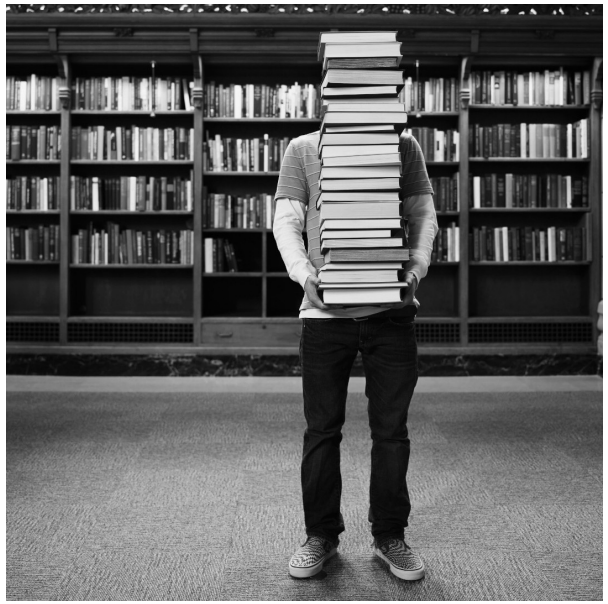
LIBRARY PARADIGM

Assume you are writing a paper

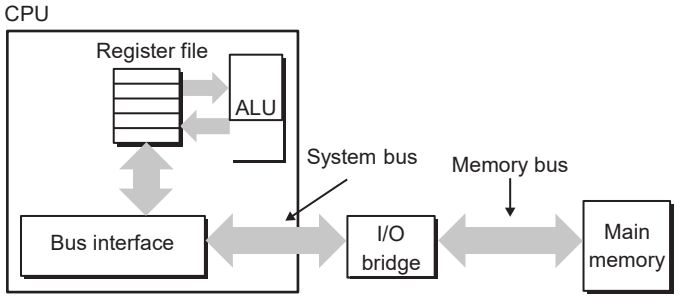
You go to the local library to work undisturbed

How do you organize yourself?

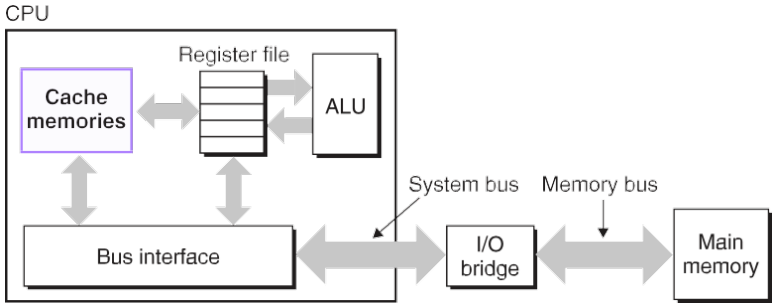
Do you constantly race back and forth in the library shelves?



CACHE MEMORIES



Classic von Neumann architecture



von Neumann architecture *with on-chip caches*

Caches are small *on-chip* memories, much like the space you have on your desk where you put your books in the library example.

CACHE MEMORIES

There are typically ***two*** level 1 caches:

- ***L1d:*** Level 1 *data* cache (exclusive for data)
- ***L1i:*** Level 1 *instruction* cache (exclusive for instructions).
  There is a separate L1 instruction cache because of different access patterns and pipelines.

The level 2 and 3 caches are *unified*. They can cache either *data or instructions*.

Latencies associated with these cache levels:

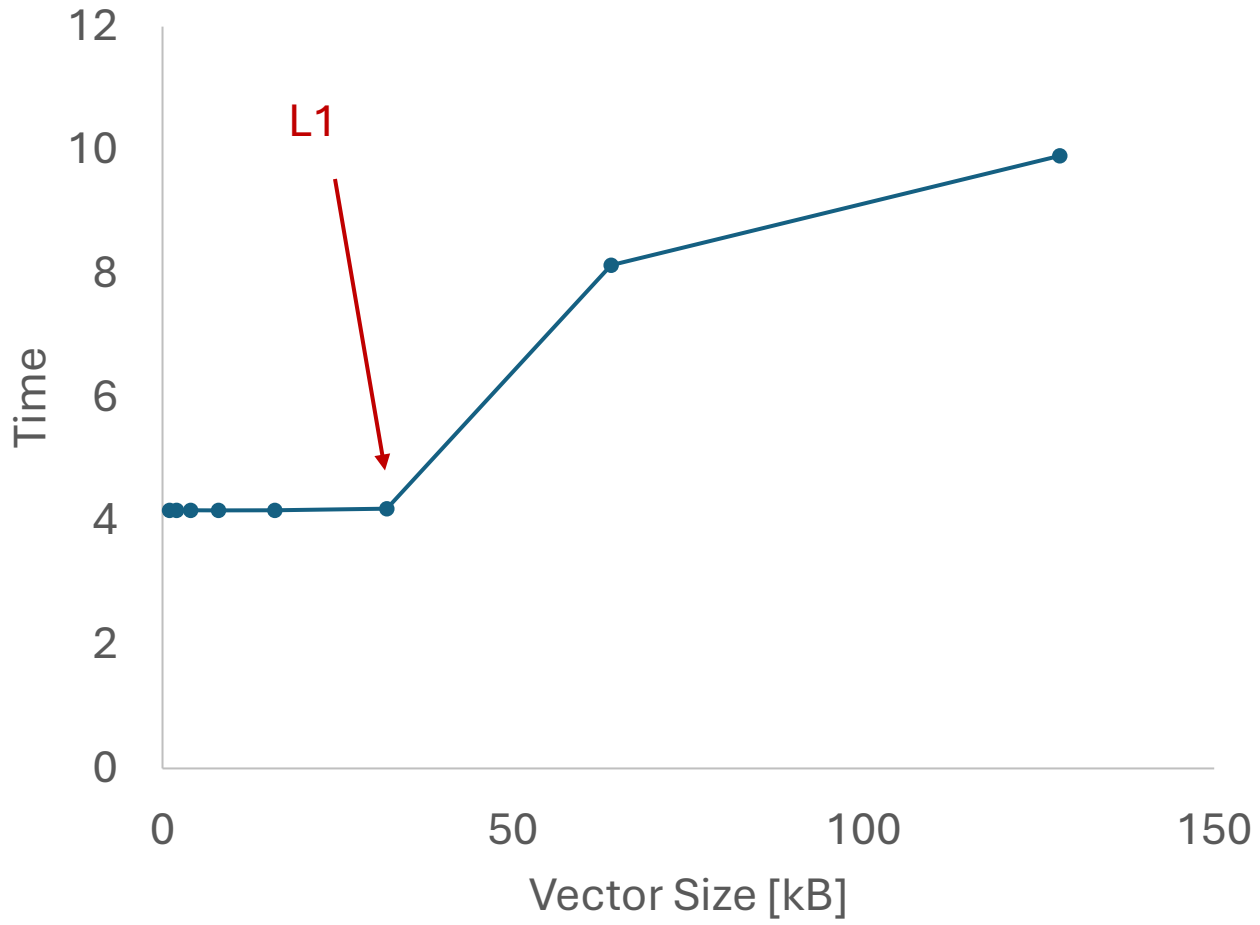
Type	Where cached	Latency (cycles)	Managed by
CPU registers	On-chip CPU registers	0	Compiler
L1 cache	On-chip L1 cache	4	Hardware
L2 cache	On-chip L2 cache	10	Hardware
L3 cache	On-chip L3 cache	50	Hardware

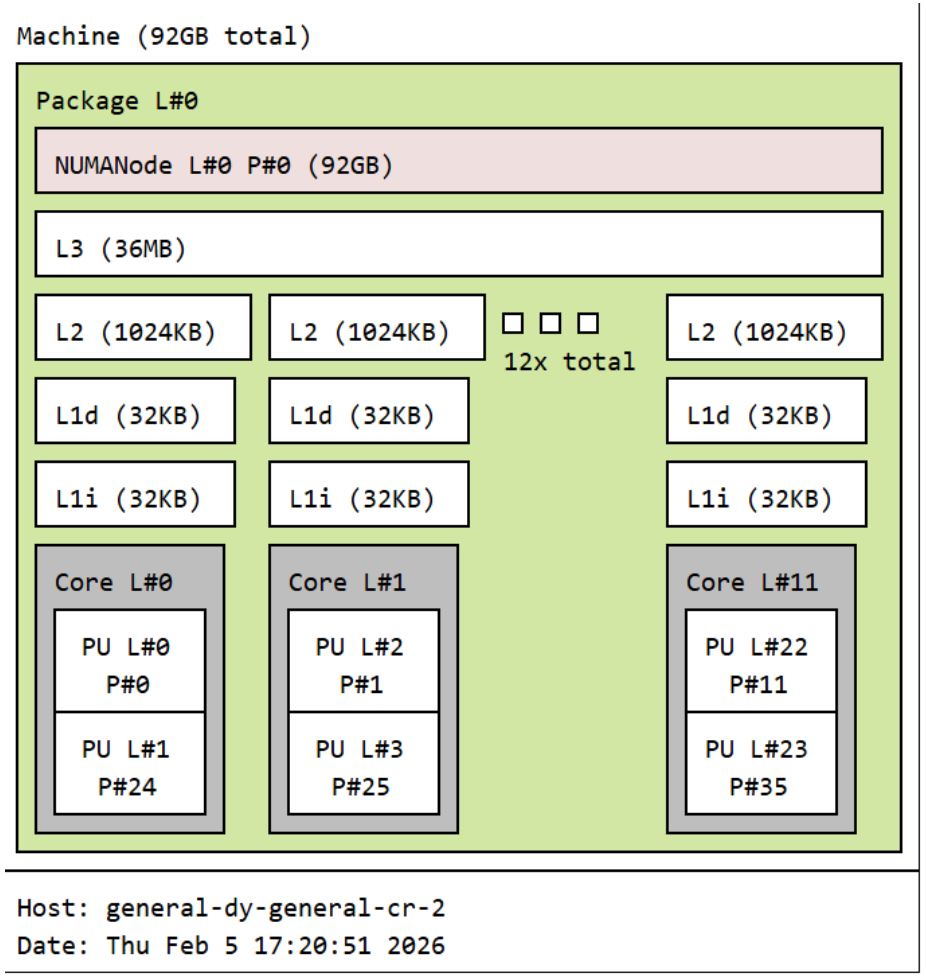
MORE LATENCY NUMBERS

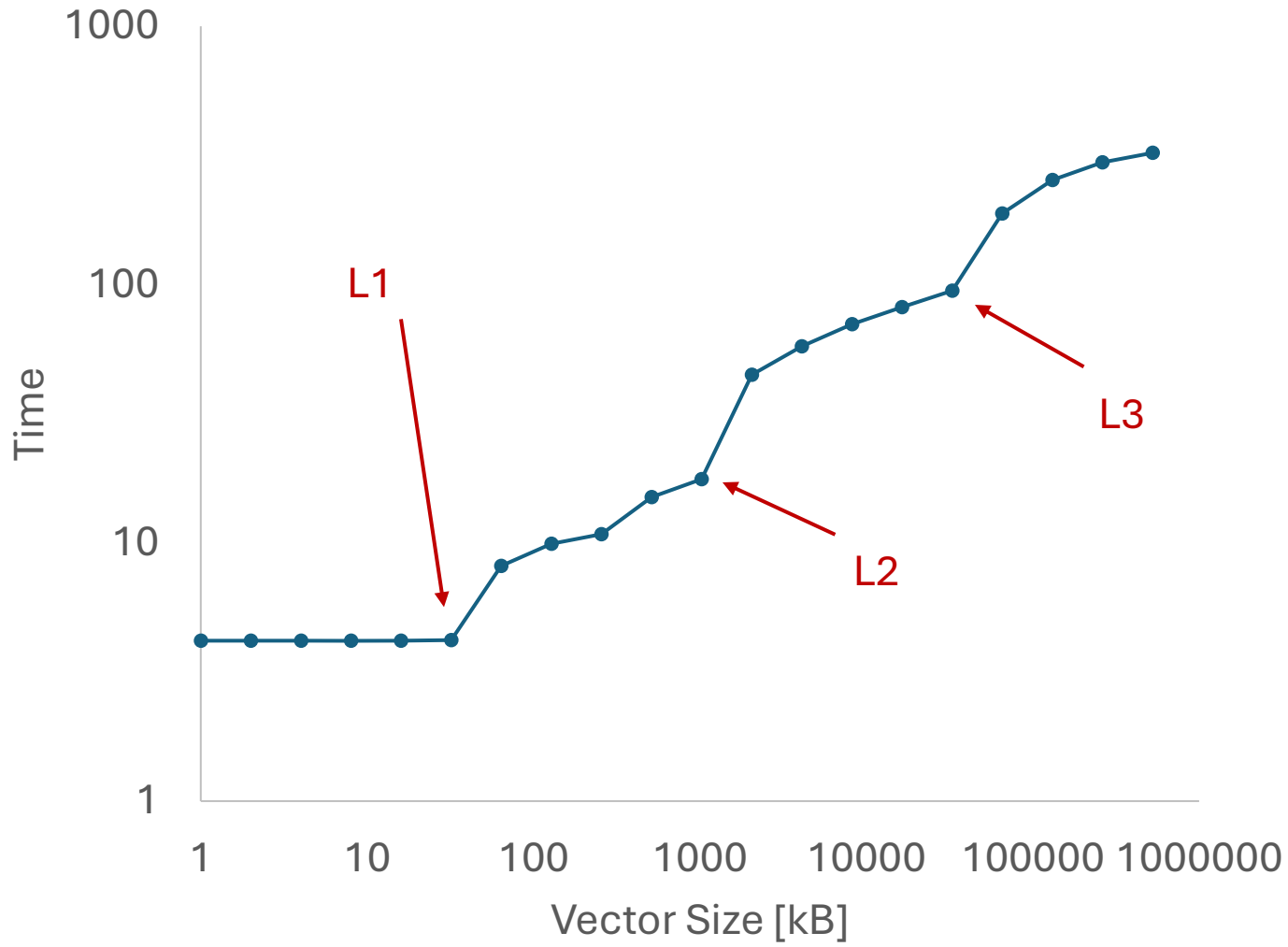
Type	Where cached	Latency (cycles)	Managed by
CPU registers	On-chip CPU registers	0	Compiler
L1 cache	On-chip L1 cache	4	Hardware
L2 cache	On-chip L2 cache	10	Hardware
L3 cache	On-chip L3 cache	50	Hardware
Virtual memory	Main memory (DRAM)	200	Hardware + OS
Disk cache	Disk controller	100'000	Controller firmware
Browser cache	Local disk	10'000'000	Web browser
Web cache	Remote server disks	1'000'000'000	Web proxy server

EXAMPLE 2

CAN WE DETECT THE CACHES EMPIRICALLY?




Machine (92GB total)
Package L#0
NUMANode L#0 P#0 (92GB)
L3 (36MB)
L2 (1024KB)
L2 (1024KB)
12x total
L2 (1024KB)
L1d (32KB)
L1d (32KB)
L1d (32KB)
L1i (32KB)
L1i (32KB)
L1i (32KB)
Core L#0
PU L#0 P#0
PU L#1 P#24
Core L#1
PU L#2 P#1
PU L#3 P#25
Core L#11
PU L#22 P#11
PU L#23 P#35
Host: general-dy-general-cr-2
Date: Thu Feb 5 17:20:51 2026

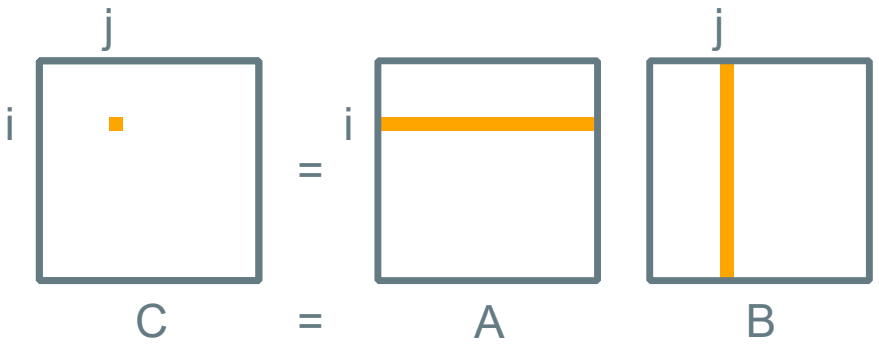

1000
100
10
1
Time
L1
L2
L3
1
10
100
1000
10000
100000
1000000
Vector Size [kB]

EXAMPLES OF LOCALITY

The matrix-matrix multiplication $C = AB$ can be computed in different ways. Assume all matrices are stored in row-major order and there is no cache (for now). How does the temporal and spatial locality differ from the following two implementations:

Implementation 1

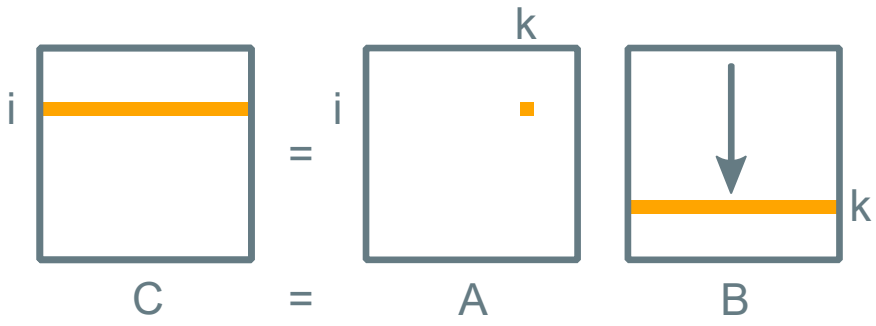
```
for (int i = 0; i < N; ++i) {
    for (int j = 0; j < N; ++j) {
        for (int k = 0; k < N; ++k) {
            C[i][j] += A[i][k] * B[k][j]
        }
    }
}
```



Computes the dot-product in the inner-most loop

Implementation 2

```
for (int i = 0; i < N; ++i) {
    for (int k = 0; k < N; ++k) {
        for (int j = 0; j < N; ++j) {
            C[i][j] += A[i][k] * B[k][j]
        }
    }
}
```

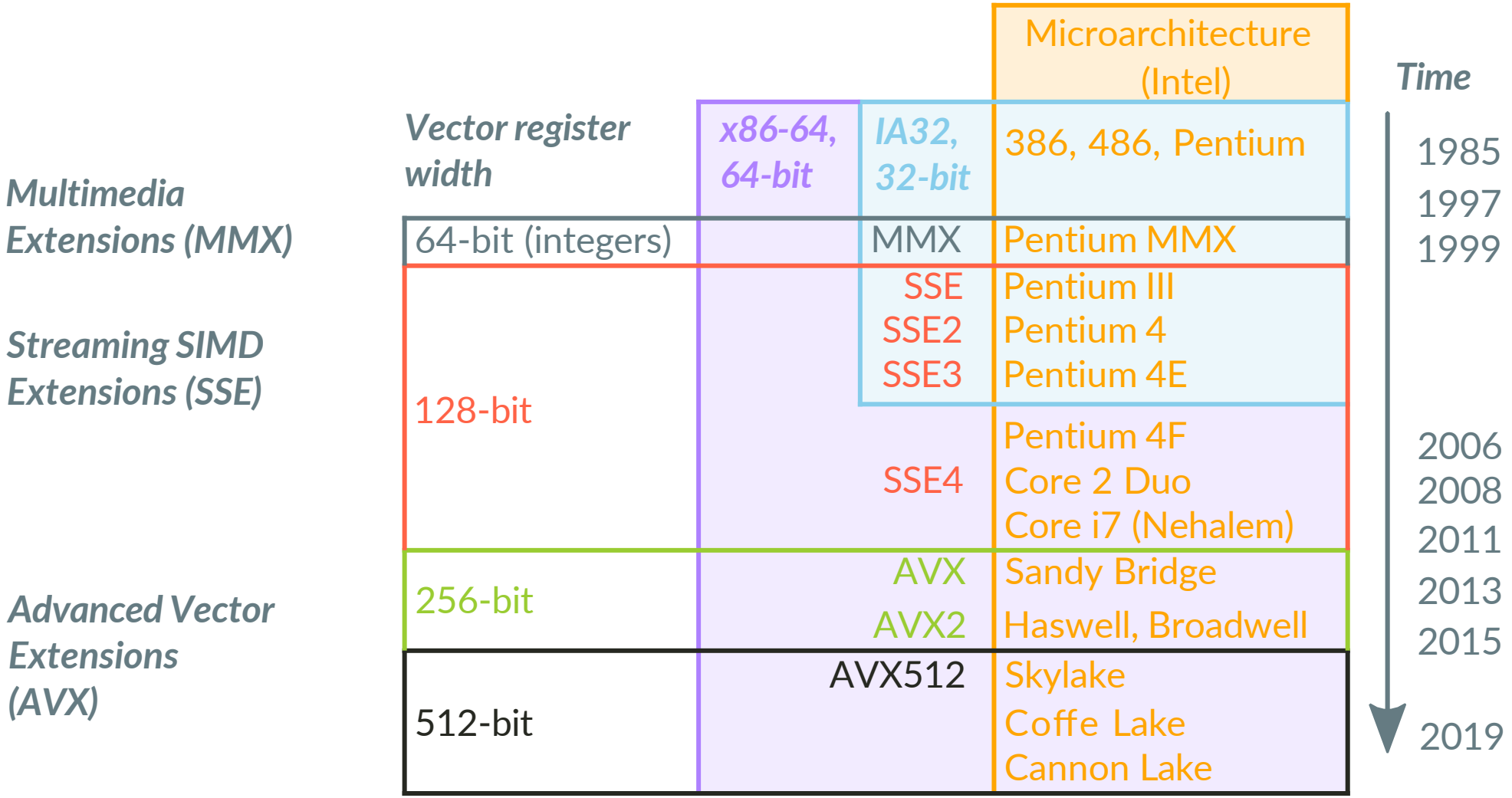


Updates rows of C by scaling rows of B with elements of A

EXAMPLE 3

SIMD VECTORIZATION

OVERVIEW OF X86 SIMD EXTENSIONS



Run `lscpu` to get a listing of what your processor supports.

LSCPU

```
[CS-2050] [wcw398@general-dy-general-cr-3 example-3]$ lscpu
Architecture:                    x86_64
  CPU op-mode(s):                32-bit, 64-bit
  Address sizes:                 46 bits physical, 48 bits virtual
  Byte Order:                    Little Endian
CPU(s):                          48
  On-line CPU(s) list:           0-47
Vendor ID:                       GenuineIntel
  Model name:                    Intel(R) Xeon(R) Platinum 8275CL CPU @ 3.00GHz
    CPU family:                  6
    Model:                       85
    Thread(s) per core:          2
    Core(s) per socket:          24
    Socket(s):                   1
    Stepping:                    7
    BogoMIPS:                    5999.99
    Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht
                                 dtscp lm constant_tsc arch_perfmon rep_good nopl xtopology nonstop_tsc cpuid aperfmperf tsc_known_freq
                                  ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hyp
                                 dnowprefetch cpuid_fault pti fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid mpx avx512f avx512dq
                                 shopt clwb avx512cd avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves ida arat pku ospke avx512_vnni
```

EXAMPLE 4

EXAMPLE 5

EXPERIMENTAL!
DIRECT SSH ACCESS TO CLUSTER

- Requires the Harvard network (Harvard VPN documentation)

- Requires ssh key
  - You already have one; see `~/.ssh/` on the cluster
  - Copy this (private) key to your local machine, and likely rename it
  - Treat this key like a password! Let us know if it is accidentally shared
  - Set permissions with `chmod 600`

- Example `~/.ssh/config` entry

EXPERIMENTAL!
DIRECT SSH ACCESS TO CLUSTER

- Example `~/.ssh/config` entry

```
Host CS-2050
  Hostname academ-acade-u490GFH4ghvP-2726a85b0a9f62fa.elb.us-east-1.amazonaws.com
  user wcw398
  IdentityFile ~/.ssh/CS-2050_id_rsa
```